

- [实验目的](#)
- [预习内容](#)
- [Docker 和 Kubernetes 简介](#)
 - [名词解释](#)
- [实验内容](#)
 - [任务一：在RDK上安装 Docker](#)
 - [1. 安装 Docker](#)
 - [2. 解决时间同步问题](#)
 - [3. 测试 Docker 时间同步](#)
 - [4. 非 root 用户使用 Docker](#)
 - [任务二：编写 Dockerfile 并构建 Docker 镜像](#)
 - [任务三：使用 Docker 运行打地鼠代码](#)
 - [附：常用 Docker 命令](#)
 - [任务四：设置 Kubernetes 集群并部署完整版打地鼠程序](#)
 - [步骤一：安装 k3s](#)
 - [准备工作：检查 VPN 连接](#)
 - [在云服务器上：安装 k3s server](#)
 - [在每个RDK上：安装 k3s agent](#)
 - [步骤二：同步 Docker 镜像](#)
 - [在云服务器上：搭建 Docker registry](#)
 - [在每个RDK上：允许不安全的 registry](#)
 - [在构建镜像用的RDK上：将镜像推送到 registry](#)
 - [步骤三：部署应用](#)
 - [创建 Kubernetes 部署文件](#)
 - [设置服务器节点的 Taint](#)
 - [修改打地鼠程序以自动设置 Agent ID](#)
 - [部署应用](#)
 - [附：常用 Kubernetes 命令](#)
- [验收要求](#)

实验目的

- 熟悉主流容器化技术，如 Docker 和 Kubernetes 的基本概念和使用方法。
- 学习如何使用 Docker 封装应用程序，并在 Kubernetes 集群中部署应用程序。

预习内容

1. 阅读实验指导书，熟悉实验流程及术语。
2. 进行任务二中所需的代码简化工作，以便快速测试 Docker 的单机部署。
3. 进行任务四步骤三中的代码修改工作，使得打地鼠程序能够自动从环境变量中读取和设置 Agent ID。

Docker 和 Kubernetes 简介

在实验四中，我们使用了他人提供的 Docker 镜像来运行 EMQX broker 服务器应用。Docker 是一个开源平台，可以使应用与底层系统隔离，简化应用的部署和扩展。使用 Docker，开发者可以将应用及其依赖项打包进一个可移植的容器中，这个容器可以在任何支持 Docker 的系统上运行。

Kubernetes（简称 k8s）是一个自动部署、扩展和管理容器化应用程序的系统。它支持多个 Docker 容器的协调运行，可以在实际生产环境中实现容器的高可用性和负载均衡。

在本次实验中，我们的目标是构建自己的 Docker 镜像，以封装我们自己开发的打地鼠游戏。使用 Docker 和 Kubernetes 的显著优势之一是简化了在多个设备上部署相同应用程序的过程。在我们的RDK实验中，该优势具体体现为：

1. **一致性和可复制性：**通过构建自己的 Docker 镜像，我们可以确保在所有三个RDK上运行的程序完全相同。这种一致性来自于 Docker 容器内部环境的隔离性和独立性，因此容器内的应用程序运行不会受到宿主机系统配置差异的影响。
2. **简化部署过程：**Kubernetes 能够自动化容器的部署和管理。一旦设置了 Kubernetes 集群，我们可以使用一个简单的命令或者配置文件，轻松地在所有连接的RDK上部署相同的 Docker 镜像。这意味着无需手动在每个设备上设置环境和配置，极大地提高了部署效率。
3. **易于扩展和更新：**随着实验的发展，如果需要更新或改进应用程序，我们只需要更新 Docker 镜像，并重新部署到 Kubernetes 集群。Kubernetes 可以自动管理更新，逐个更新每个节点上的容器，确保所有设备都能运行最新版本的应用程序。

名词解释

- Docker 相关：

- **镜像 (Image)**：Docker 镜像是一个只读的模板，用于创建 Docker 容器。镜像包含了运行应用程序所需的所有文件和依赖项，可以在不同的 Docker 容器之间共享和复制。
 - **容器 (Container)**：Docker 容器是 Docker 镜像的运行实例。容器是一个独立的、隔离的运行环境，可以在其中运行应用程序。容器之间相互隔离，但又共享主机系统的内核。
 - **注册表服务器 (Registry)**：Docker 镜像的存储和分发服务。Docker Hub 是 Docker 官方提供的注册表服务器，用于存储和分享 Docker 镜像。除了 Docker Hub，还有其他公共和私有的注册表服务器供用户选择，也可以搭建自己的注册表服务器。
- Kubernetes 相关：
 - **Pod**：Kubernetes 中最小的调度单元。Pod 是一个或多个容器的组合，共享网络和存储资源，通常用于部署一个应用程序。在我们的实验中，每个 Pod 只运行一个容器。
 - **节点 (Node)**：集群中的一个节点，可以是物理机器或虚拟机。每个节点上可以运行若干个 Pod。在我们的实验中，云服务器是一个节点，每个RDK也是一个节点；每个RDK节点上运行一个 Pod。
 - **部署 (Deployment)**：Kubernetes 中用于定义和管理 Pod 的对象。Deployment 可以指定 Pod 的副本数量、容器镜像、资源限制等信息，以确保 Pod 能够按照预期的方式运行。
 - **Node affinity 及 Pod affinity**：Kubernetes 的调度策略之一，用于指定 Pod 应该运行在哪些节点上。Node affinity 可以根据节点的标签 (Label) 来选择节点，以满足特定的调度需求。Pod affinity 则是根据其他 Pod 的标签来选择节点，实现 Pod 之间的相容或互斥关系。
 - **Taint**：Kubernetes 的调度策略之一，用于标记节点的特性。Taint 可以阻止 Pod 被调度到某些节点上，除非 Pod 明确声明了对应的容忍 (Toleration) 规则。
- **Containerd**: 是 Docker 背后的核心组件之一，专责于容器的生命周期管理，这包括容器的创建、运行、停止和删除等操作。Containerd 被设计为 Docker 的底层模块，但它是独立于 Docker 的，可以被视为一个通用的容器运行时。Docker 和 Kubernetes 都可以通过调用 Containerd 的 API 来管理容器。这种架构不仅确保了系统的模块化和高效性，同时也支持更广泛的技术生态系统，使得 Containerd 能够适应更多种的运行环境和云平台。

实验内容

任务一：在RDK上安装 Docker

1. 安装 Docker

使用官方脚本安装 Docker：

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

官方安装脚本不支持Raspbian Buster系统，因此采用清华源安装Docker

```
sudo vim /etc/apt/sources.list
```

替换为

```
deb http://mirrors.tuna.tsinghua.edu.cn/raspbian/raspbian/ buster main non-free contrib
deb-src http://mirrors.tuna.tsinghua.edu.cn/raspbian/raspbian/ buster main non-free cont
```

安装依赖

```
# 安装依赖工具
sudo apt update
sudo apt install -y apt-transport-https ca-certificates curl gnupg2

# 添加 Docker 的 GPG 密钥
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg

# 添加 Docker 清华源
echo "deb [arch=armhf signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://"
```

安装Docker

```
# 更新软件包索引
sudo apt update

# 安装 Docker CE
sudo apt install -y docker-ce docker-ce-cli containerd.io
```

```
# 验证安装
sudo docker run --rm hello-world
```

如果验证安装步骤报错，则需要配置镜像加速，具体参考实验4指导书

2. 解决时间同步问题

时间不同步是一个已知问题，详情可以参见 [GitHub 讨论](#)。以下是修复步骤：

1. 添加 "testing" 仓库到 `/etc/apt/sources.list`：

```
echo "deb http://raspbian.raspberrypi.org/raspbian/ testing main" | sudo tee -a /etc/
```

2. 设置 "testing" 仓库的优先级较低，以确保默认情况下仍从主仓库安装软件包。创建一个优先级配置文件：

```
echo 'Package: *
Pin: release a=testing
Pin-Priority: 150' | sudo tee /etc/apt/preferences.d/testing
```

3. 更新软件包索引并安装更新的 `libseccomp2`：

```
sudo apt update
sudo apt install libseccomp2/testing
```

如果遇到libseccomp2报错，可按照如下方式安装

```
wget http://archive.raspbian.org/raspbian/pool/main/libs/libseccomp/libseccomp2_2.5.4-1%
sudo dpkg -i libseccomp2_2.5.4-1+rpi1+deb12u1_armhf.deb
```

3. 测试 Docker 时间同步

运行一个基于 Ubuntu 的容器来检查时间同步是否正确：

```
sudo docker run --rm ubuntu date
```

比较这个时间与RDK主机上的时间（使用 `date` 命令）来验证是否修复了时间同步问题。

4. 非 root 用户使用 Docker

为了让非 root 用户（例如默认的 `pi` 用户）可以运行 Docker 命令，需要将用户添加到 `docker` 组：

```
sudo usermod -aG docker pi
```

然后，退出 SSH 会话并重新登录，以使组更改生效。

任务二：编写 Dockerfile 并构建 Docker 镜像

此任务中，我们将把打地鼠程序封装到一个 Docker 镜像中。简便起见，只需在一台RDK上进行实验即可。在进行实验前，可适当简化程序，保证程序只需要一个RDK节点即可运行，并能够验证读取传感器数据、控制 LED 灯、和云服务器通信等功能均正常运行。

1. 在实验五的目录中存放打地鼠游戏所需 Python 代码：

```
mkdir -p experiment5/src
cd experiment5/src
cp -R ../experiment4/* .    # 修改路径以适应实际情况、复制打地鼠游戏所需代码
```

2. 编写入口脚本：

```
vim entrypoint.sh
```

填入以下内容：

```
#!/bin/bash
python3 main.py    # 替换为运行打地鼠游戏所需的命令
```

为脚本添加可执行权限：

```
chmod +x entrypoint.sh
```

3. 在 `exp5` 目录中创建 Dockerfile：

```
cd experiment5/
```

创建 Dockerfile，注意没有文件后缀：

```
vim Dockerfile
```

内容如下：

```
FROM python:3.7

# 设置工作目录
WORKDIR /app

# 安装 Python 依赖
# 在下一行中，按需添加你的程序所需的依赖包
RUN pip3 install -i https://pypi.tuna.tsinghua.edu.cn/simple RPi.GPIO gpiozero smbus

# 安装 QUIC 依赖
# 在执行下一行前，确保将实验四中的 libmsquic.so.2 文件拷到 Dockerfile 所在目录
COPY libmsquic.so.2 /usr/lib/

# 复制代码
COPY src/ /app/

# 运行 Python 脚本
CMD ["bash", "entrypoint.sh"]
```

4. 构建 Docker 镜像：

```
sudo docker build -t exp5:latest .
```

注：修改代码后，需要重新构建 Docker 镜像。 构建过程不会从头开始，而是会利用 Docker 缓存机制，只重新构建修改过的部分，因此不会花费太多时间。

任务三：使用 Docker 运行打地鼠代码

运行你的 Docker 容器，确保映射必要的设备文件：

```
sudo docker run --rm --privileged --name exp5 --device /dev/i2c-1:/dev/i2c-1 exp5:latest
```

命令拆解：

- `docker run`：这是 Docker 的一个基本命令，用于启动一个新的容器实例。
- `--rm`：此选项指示 Docker 在容器退出时自动删除容器。这有助于避免在运行多个容器实例时产生大量无用的容器。
- `--privileged`：此选项赋予容器访问主机所有设备的权限。这在需要直接控制硬件（如 GPIO 或 I2C 设备）时非常有用。
- `--name exp5`：使用 `--name` 选项为运行的容器指定一个名称，这里的名称是 `exp5`。指定名称后，可以通过这个名称来引用或管理容器，而不是使用容器的 ID。
- `--device /dev/i2c-1:/dev/i2c-1`：这个选项将主机上的 `/dev/i2c-1` 设备文件映射到容器内的同一路径。这对于需要使用 I2C 通信协议的设备（如 APDS-9960 传感器）是必需的，以确保容器内的应用能够正确访问和控制这些设备。
- `exp5:latest`：这指定了容器要使用的 Docker 镜像，亦即我们刚刚构建的镜像。`exp5` 是镜像的名称，`latest` 是镜像的标签。

附：常用 Docker 命令

- `docker ps`：列出当前正在运行的容器。
- `docker ps -a`：列出所有容器，包括已经停止的容器。
- `docker stop <container-id>`：停止指定 ID 的容器。
- `docker rm <container-id>`：删除指定 ID 的容器。

- `docker images` : 列出所有本地镜像。
- `docker rmi <image-id>` : 删除指定 ID 的镜像。
- `docker exec -it <container-id> bash` : 进入正在运行的容器的 shell 环境。
- `docker logs <container-id>` : 查看容器的日志输出。

任务四：设置 Kubernetes 集群并部署完整版打地鼠程序

在本任务中，我们将使用 Kubernetes（简称 k8s，因为 k 和 s 之间有 8 个字母）来部署我们的打地鼠程序。Kubernetes 是一个开源的容器编排引擎，可以自动化容器的部署、扩展和管理。在我们的实验中，Kubernetes 将负责在所有 RDK 节点上运行我们的 Docker 镜像，确保所有设备都能运行相同的打地鼠程序。具体而言，我们将使用 k3s，这是一个专为 IoT 设备和边缘计算场景设计的轻量级 Kubernetes 发行版。

步骤一：安装 k3s

准备工作：检查 VPN 连接

沿用[实验四](#)中的设置，在云服务器和各个 RDK 节点上通过 `sudo wg-quick up wg0` 命令启动 WireGuard VPN 连接。在各个节点之间运行 `ping 10.0.0.x` 命令（其中 x 为节点编号）来检查 VPN 连接是否正常。

在云服务器上：安装 k3s server

1. 通过中国镜像安装 k3s

使用中国内地的镜像安装 k3s，可以加快下载速度并提高安装成功率。在服务器上运行以下命令：

```
curl -sfl https://rancher-mirror.oss-cn-beijing.aliyuncs.com/k3s/k3s-install.sh | INS
```

如上述安装方式不成功，可通过离线方式安装

```
wget https://rancher-mirror.rancher.cn/k3s/k3s-install.sh
wget https://rancher-mirror.rancher.cn/k3s/v1.30.2-k3s2/k3s
sudo chmod a+x ./k3s ./k3s-install.sh
sudo cp ./k3s /usr/local/bin
INSTALL_K3S_MIRROR=cn K3S_NODE_NAME=server INSTALL_K3S_SKIP_DOWNLOAD=true ./k3s-inst
```

2. 设置节点 IP 地址

由于云服务器可能拥有多个 IP 地址（公网 IP、阿里云内网 IP、VPN 内网 IP 等），我们需要设置 k3s server 使用 VPN 内网 IP 作为节点地址，以便 RDK 节点能够连接到 k3s server。编辑服务器上的 k3s 配置文件 `/etc/systemd/system/k3s.service`，在末尾 `ExecStart` 开头的设置中添加 `--node-ip` 和 `--advertise-address` 选项，如下所示：

```
ExecStart=/usr/local/bin/k3s \
server \
--node-ip 10.0.0.1 \
--advertise-address 10.0.0.1 \
```

编辑完成后，重启 k3s 服务以应用更改：

```
sudo systemctl daemon-reload      # 重新加载 systemd 配置
sudo systemctl restart k3s         # 重启 k3s 服务
```

3. 检查 k3s 服务状态

安装完成后，检查 k3s 服务是否正常运行：

```
kubectl get nodes
```

如果 k3s server 正常运行，应该能看到一个节点（即服务器本身）处于 `Ready` 状态，如下所示：

```
# root @ iZ2zebo2506v0ea1r33p0fZ in ~ [22:53:54]
$ kubectl get nodes
NAME      STATUS    ROLES          AGE    VERSION
server    Ready     control-plane,master  35m    v1.29.3+k3s1
```


4. 查看 k3s 服务 Token

为了让其他节点加入到 k3s 集群，我们需要获取 k3s 服务的 Token。运行以下命令查看 Token：

```
sudo cat /var/lib/rancher/k3s/server/node-token
```

记录下输出的 Token，后续将用于RDK节点的加入。

在每个RDK上：安装 k3s agent

1. 启用 cgroup

在RDK上，需要确保 cgroup 功能被启用以支持 Kubernetes（因为 Kubernetes 需要使用 cgroup 功能来进行资源限制和监控，确保各个容器的资源分配和管理能够有效进行）。请按照以下步骤修改启动命令行：

- 连接到RDK，并输入以下命令编辑启动文件（参考[Vim教程](#)）：

```
sudo vim /boot/cmdline.txt
```

- 在文件的行尾添加以下内容（确保整个命令在一行显示，不要换行）：

```
cgroup_memory=1 cgroup_enable=memory
```

- 保存并关闭编辑器
- 重启RDK以应用更改：

```
sudo reboot
```

2. 启用 VPN 连接

重启后，需要重新启动 VPN 连接：

```
sudo wg-quick up wg0
```

3. 通过中国镜像安装 k3s

与服务器上的安装命令类似，但需要指定服务器的地址和 Token。在RDK上运行以下命令：

```
curl -sfl https://rancher-mirror.oss-cn-beijing.aliyuncs.com/k3s/k3s-install.sh | IN
```

如上述安装方式不成功，可通过离线方式安装

```
wget https://rancher-mirror.rancher.cn/k3s/k3s-install.sh
wget https://rancher-mirror.rancher.cn/k3s/v1.30.2-k3s2/k3s-armhf
```

如果RDK上无法下载 k3s-install.sh 和 k3s-armhf ，则在本地下载后scp复制到RDK中

```
scp k3s-install.sh k3s-armhf pi@<ip>:~/
```

```
sudo chmod a+x ./k3s-armhf ./k3s-install.sh
sudo cp ./k3s-armhf /usr/local/bin
sudo mv /usr/local/bin/k3s-armhf /usr/local/bin/k3s
INSTALL_K3S_MIRROR=cn K3S_URL=https://10.0.0.1:6443 K3S_TOKEN=<server-token> K3S_NODE
```

其中， <server-token> 替换为服务器上获取的 Token， <node-name> 替换为节点的名称（如 pi1、pi2 等，不同节点之间须互不相同）；尖括号 <> 不需要输入。

4. 检查节点状态

安装完成后，检查节点是否成功加入 k3s 集群。在服务器上运行以下命令：

```
kubectl get nodes
```

如果一切正常，应该能看到新加入的节点都处于 `Ready` 状态。例如，加入了一个新节点 `pi1` 后，应该看到类似如下的输出：

```
# root @ iZ2zebo2506v0ea1r33p0fZ in ~ [22:53:54]
$ kubectl get nodes
NAME          STATUS    ROLES          AGE    VERSION
server        Ready     control-plane,master 35m    v1.29.3+k3s1
pi1           Ready     <none>          21m    v1.29.3+k3s1
```

加入全部三个RDK节点后，应该在看到所有节点都处于 `Ready` 状态。

步骤二：同步 Docker 镜像

在此步骤中，我们将在云服务器上搭建自己的 Docker registry，并将本地构建的 Docker 镜像推送到 registry。此后，所有RDK节点将从 registry 中拉取镜像，以确保所有节点运行的程序版本一致。

在云服务器上：搭建 Docker registry

Docker registry 本身也是一个 Docker 容器，可以通过 Docker Hub 上的官方镜像快速部署。在服务器上运行以下命令启动 registry 容器：

```
sudo docker run -d -p 5000:5000 --restart=always --name registry registry:2
```

此时，运行 `docker ps` 命令应该能看到 registry 容器正在运行，如下所示：

```
# root @ iZ2zebo2506v0ea1r33p0fZ in ~ [0:24:13]
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
adc05f0de012   registry:2     "/entrypoint.sh /etc..." 2 hours ago    Up 2 hours    0.0.0.0:5000->5000/tcp, :::5000->5000/tcp
94a6a7d67963   emqx/emqx:5.5.0 "/usr/bin/docker-ent..." 3 weeks ago    Up 3 weeks    4370/tcp, 0.0.0.0:1883->1883/tcp, :::1883->1883/tcp, 0.0.0.0:8083->8083/tcp, :::8083->8083/tcp, 0.0.0.0:18083->18083/tcp, :::18083->18083/tcp, 5369/tcp, 0.0.0.0:14567->14567/udp, :::14567->14567/udp
```

图中显示，云服务器上有两个容器正在运行，一个是 registry，另一个是实验四中启动的 EMQX broker。

在每个RDK上：允许不安全的 registry

由于云服务器和RDK节点之间的通信是通过 VPN 连接的，而 VPN 连接已经保证了通信的安全性，因此我们在 registry 层面不需要额外的安全措施。为了允许RDK节点拉取 registry 中的镜像，需要在每个RDK节点上修改 Docker 和 k3s 配置，允许不安全的 registry。

- 1. 编辑 Docker 配置文件 `/etc/docker/daemon.json`：

```
sudo vim /etc/docker/daemon.json
```

在文件中添加以下内容：

```
{
  "insecure-registries": ["10.0.0.1:5000"]
}
```

- 2. 编辑 k3s 配置文件 `/etc/rancher/k3s/registries.yaml`：

```
sudo mkdir -p /etc/rancher/k3s
sudo vim /etc/rancher/k3s/registries.yaml
```

在文件中写入以下内容：

```
mirrors:
  10.0.0.1:5000:
    endpoint:
      - "http://10.0.0.1:5000"
  docker.io:
    endpoint:
      - "https://registry.aliyuncs.com"
      - "https://docker.mirrors.ustc.edu.cn"
```

3. 重启 Docker 和 k3s 服务以应用更改：

```
sudo systemctl restart docker
sudo systemctl restart k3s-agent.service
```

在构建镜像用的RDK上：将镜像推送到 registry

在一台RDK上编写完整版的打地鼠程序，并构建 Docker 镜像。然后，将镜像推送到 registry 中，以便其他节点拉取。每次更新完程序后，可以重复如下步骤：

1. 构建 Docker 镜像：在 Dockerfile 所在目录运行以下命令构建镜像：

```
sudo docker build -t exp5:latest .
```

2. 标记镜像：将镜像标记为 registry 中的地址：

```
sudo docker tag exp5:latest 10.0.0.1:5000/exp5
```

3. 推送镜像：将镜像推送到 registry：

```
sudo docker push 10.0.0.1:5000/exp5
```

步骤三：部署应用

创建 Kubernetes 部署文件

在云服务器上，创建一个 Kubernetes 部署文件 `exp5.yaml`，用于部署打地鼠程序。文件内容如下：

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: exp5
spec:
  replicas: 3
  selector:
    matchLabels:
      app: exp5
  template:
    metadata:
      labels:
        app: exp5
    spec:
      tolerations:
      - key: "key1"
        operator: "Equal"
        value: "value1"
        effect: "NoSchedule"
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchExpressions:
                  - key: app
                    operator: In
                    values:
                      - exp5
              topologyKey: "kubernetes.io/hostname"
      containers:
      - name: exp5
        image: 10.0.0.1:5000/exp5:latest
        imagePullPolicy: Always
        volumeMounts:
        - mountPath: /dev/i2c-1
          name: dev-i2c-1
        securityContext:
          privileged: true
        env:
        - name: MY_POD_NAME
```



```
      valueFrom:
        fieldRef:
          fieldPath: metadata.name
    - name: MY_POD_IP
      valueFrom:
        fieldRef:
          fieldPath: status.podIP
    - name: MY_NODE_NAME
      valueFrom:
        fieldRef:
          fieldPath: spec.nodeName
  volumes:
    - name: dev-i2c-1
      hostPath:
        path: /dev/i2c-1
```

上面的文件定义的功能如下：

- 创建一个名为 `exp5` 的 Deployment，包含 3 个 Pod。
- 通过 `tolerations` 设置，确保 Pod 不会被调度到具有特定标记的节点（根据下一小节的设置，即云服务器节点）上。
- 通过 `podAntiAffinity` 设置，确保一个RDK节点上只运行一个 Pod。
- 指定容器镜像为 `exp5`，从自建的 registry 中拉取最新版本。
- 允许容器以最高权限运行（以便读写外设），并将传感器所需的 i2c 设备映射到容器中
- 通过环境变量将 Pod 名称、Pod IP 和节点名称传递给 Pod 中运行的程序。

注：先前示例代码中使用的 `RPi.GPIO` 库在由 k3s 创建的容器中运行时可能报错。实际上不需要使用 `RPi.GPIO` 而只需要 `apds9960` 库即可正常读取传感器；可相应地修改代码。

设置服务器节点的 Taint

为了确保用于RDK运行的 Pod（即打地鼠的 agent 程序）不会被调度到服务器节点上，我们需要在服务器节点上设置一个 Taint。在服务器上运行以下命令：

```
kubectl taint nodes server key1=value1:NoSchedule
```

修改打地鼠程序以自动设置 Agent ID

根据上述部署文件，三个RDK节点将根据完全相同的镜像运行打地鼠程序。为了区分不同的节点，我们需要在程序中自动设置 Agent ID。这可以通过读取由 k3s 传入 Pod 的环境变量 `MY_POD_NAME`、`MY_POD_IP` 或 `MY_NODE_NAME` 来实现。

通过 Python 读取环境变量的示例代码如下：

```
import os

pod_name = os.getenv("MY_POD_NAME")
pod_ip = os.getenv("MY_POD_IP")
node_name = os.getenv("MY_NODE_NAME")

print(f"Running on pod {pod_name} at IP {pod_ip} on node {node_name}")
```

请参考上述代码，自行设计修改打地鼠程序的方法，使其能够自动设置 Agent ID。

部署应用

经过上述准备工作，我们终于可以部署打地鼠程序了！在云服务器上运行以下命令：

```
kubectl apply -f exp5.yaml
```

该命令将自动创建 Deployment，并在所有RDK节点上运行打地鼠程序。运行以下命令查看所有 Pod 的状态：

```
kubectl get pods -o wide
```

下图展示了只有一个RDK节点 `pi1` 在线的情况下的输出：

```
# root @ iZ2zebo2506v0ea1r33p0fZ in ~ [3:01:39]
$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
exp5-6988d6c9dc-jts7q	0/1	Pending	0	25s	<none>	<none>	<none>		<none>	
exp5-6988d6c9dc-w2tbp	1/1	Running	0	25s	10.42.1.14	pi1	<none>		<none>	

若三个RDK节点都在线，待所有容器创建完成后，除了表头外应该能看到三行输出：

- STATUS 均为 `Running`，表示所有 Pod 正在运行。
- NODE 列显示 Pod 所在的节点名称，应该分别为 `pi1`、`pi2` 和 `pi3`。
- IP 列显示 Pod 的 IP 地址。注意这里的 IP 地址是由 k3s 创建的虚拟网络中的地址。

观察三台RDK，是不是已经开始打地鼠了呢？如果一切正常，恭喜你，你已经成功部署了一个分布式的打地鼠游戏！否则，在调试和修改代码后，可以按照下面的步骤重新部署应用：

- 在一台RDK上修改代码、重新构建镜像、重新执行标记和推送镜像的步骤。
- 重新部署：运行命令 `kubectl delete -f exp5.yaml` 删除旧的 Deployment，然后再次运行 `kubectl apply -f exp5.yaml` 部署新的应用。

附：常用 Kubernetes 命令

- `kubectl get pods`：列出所有 Pod。
- `kubectl get nodes`：列出所有节点。
- `kubectl describe pod <pod-name>`：查看指定 Pod 的详细信息。
- `kubectl logs <pod-name>`：查看指定 Pod 的日志输出。
- `kubectl exec -it <pod-name> -- /bin/bash`：进入指定 Pod 的 shell 环境。
- `kubectl apply -f <yaml-file>`：创建或更新指定的 Deployment、Service 等 Kubernetes 对象。
- `kubectl delete -f <yaml-file>`：删除指定的 Deployment、Service 等 Kubernetes 对象。

验收要求

1. 通过 `docker run` 命令运行单机简化打地鼠程序
2. 搭建 k3s 集群，通过 `kubectl get pods` 命令查看所有服务状态
3. 在 k3s 集群中运行打地鼠游戏
4. 通过 `docker stats --no-stream --format '{{.Container}}: {{.MemUsage}}'` 命令查看在 Docker 中运行的打地鼠程序的内存占用，与不使用 Docker 直接运行打地鼠程序时的内存占用进行对比（后者可以用 `htop` 命令查看）。回答以下问题：使用 Docker 运行程序造成的额外内存开销有多大？

选做任务：

1. 编写一个收集游戏状态和分数的程序，将其封装为一个镜像、其 Pod 在云服务器节点上运行
2. 自学 Kubernetes 的 Service 功能，将选做任务 1 中收集的信息显示在一个 Web 页面上、并通过 Service 允许公网访问

Comments